

State of the Art in Open-Source Matrix Protocol Implementations: Specifications for GaussMatrix and GaussInteract

GaussMatrix / GaussInteract Platform Engineering Group

Abstract—The Matrix protocol has become the leading open standard for federated, end-to-end-encrypted real-time communication, underpinning national e-government deployments and large private federations. Selecting an implementation as the baseline for a new product is, however, hampered by the absence of a systematic, reproducible comparison framework. This paper presents (i) a systematic survey of all actively maintained open-source Matrix *homeservers* written in Rust — Conduit, Continuuity, Tuwunel, and Grapevine—and of the principal actively maintained *cross-platform clients*—the Element family (Element X on the Matrix Rust SDK), FluffyChat, Cinny, and SchildiChat—and (ii) a rigorous five-dimensional benchmark framework with an explicit weighted scoring model for selecting a server baseline for GAUSSMATRIX and a client baseline for GAUSSINTERACT. Each project is characterised by architecture, modules, specification compliance, storage and federation support, scalability design, limitations, and community-health metrics, and is related to the peer-reviewed literature on Matrix scalability and end-to-end-encryption security. The benchmark scores performance, scalability, specification compliance, operational maturity, and code quality for servers, and platform coverage, feature completeness, developer experience, performance, and community longevity for clients. Applying the weighted model, Tuwunel is selected as the server baseline (aggregate 8.88/10) and the Element X / Matrix Rust SDK stack as the client baseline (aggregate 9.03/10), with FluffyChat identified as the strongest single-codebase alternative. A sensitivity analysis confirms that both selections are stable across a wide range of weight assignments. We close with integration considerations, risks, and a roadmap for extending each baseline.

Index Terms—Matrix protocol, federated messaging, homeserver, Rust, end-to-end encryption, cross-platform clients, software benchmarking, systematic survey, decentralised communication.

I. INTRODUCTION

DECENTRALISED and federated communication has moved from a niche concern to a strategic requirement for governments and enterprises that need sovereignty over their messaging data. Among the candidate standards, the *Matrix* protocol—an open specification for interoperable, federated, end-to-end-encrypted real-time communication—has achieved the broadest production adoption, reporting in excess of 115 million users on public, data-reporting homeservers as of late 2024 and serving as the basis for state communication systems in France, Germany, Sweden, and Luxembourg [7], [1]. Its appeal stems from a deliberate design analogy with

e-mail: any organisation can operate a server (a *homeserver*), and users on different homeservers interoperate transparently over *federation* [4], [1].

A. Motivation

Two technical trends make the present moment particularly consequential for implementers. First, the reference homeserver, Synapse, is written in Python with performance-critical components progressively rewritten in Rust [25]; this has catalysed an ecosystem of *from-scratch* Rust homeservers that promise lower memory footprints and higher throughput on commodity hardware [10], [12]. Rust is attractive here because it provides memory and data-race safety without a garbage collector, eliminating by construction the class of memory-safety defects that dominates security vulnerabilities in systems software [8]. Second, the client side is undergoing a comparable consolidation: the historically platform-specific Element applications are being superseded by *Element X*, a family of native clients built on a single shared Rust core, the Matrix Rust SDK [15], [20]. For an organisation building a high-performance homeserver product (GAUSSMATRIX) and a cross-platform client product (GAUSSINTERACT), choosing the right open-source baseline is therefore a decision with multi-year consequences.

B. Gap

Despite the importance of the choice, the literature offers no systematic, reproducible benchmark for selecting among Rust homeservers or among cross-platform Matrix clients. The peer-reviewed corpus on Matrix is concentrated on two themes: the *scalability* of federation and the event-graph replicated data type [4], [5], and the *security* of the cryptographic core, including practically-exploitable vulnerabilities and subsequent formal analyses [2], [3], [7]. Empirical ecosystem measurements have characterised the network’s (de)centralisation [6]. None of these works compares concrete implementations against operational, code-quality, and developer-experience criteria, nor do they provide a weighted decision procedure. Practitioner resources such as the Matrix ecosystem directory enumerate projects [27], [28] but stop short of a scored, weighted ranking with sensitivity analysis.

C. Contributions

This paper makes the following contributions:

- 1) A *systematic survey* of the four actively maintained open-source Matrix homeservers written in Rust (Conduit, Continuity, Tuwunel, Grapevine), each with a publication-quality architecture diagram, a compliance and storage profile, and community-health metrics (Section III).
- 2) A *systematic survey* of the principal actively maintained cross-platform clients (the Element/Element X family, FluffyChat, Cinny, SchildiChat) with a platform-coverage matrix and an SDK-centric architectural comparison (Section IV).
- 3) A *reproducible, five-dimensional benchmark framework* with an explicit weighted scoring model and justified weights for both the server- and client-selection problems (Section VI).
- 4) A *scored ranking with radar visualisations, statistical justification, and sensitivity analysis*, yielding a defensible baseline selection for GAUSSMATRIX and GAUSS-INTERACT (Sections VII and VIII).

D. Organisation

Section II reviews the Matrix protocol, the case for Rust, and enterprise client requirements. Sections III and IV present the server and client surveys. Section VI defines the benchmark framework and scoring model; Section VII reports and analyses the results. Section VIII gives recommendations, risks, and a roadmap, and Section IX concludes.

II. BACKGROUND

A. The Matrix Protocol

Matrix is specified as a set of HTTP/JSON APIs and a federation protocol governed by The Matrix.org Foundation [1]; the standard was first introduced in 2014 and has since grown into a large public federation [29]. The unit of conversation is a *room*, which is not hosted on any single server: every participating homeserver holds an equal replica of the room’s state, and servers synchronise by exchanging *events* [4]. Four API surfaces structure an implementation. The *Client–Server* (CS) API is used by clients to send and receive events, manage devices, and perform end-to-end-encryption (E2EE) key operations. The *Server–Server* (SS) API—federation—transports events between homeservers, authenticates them using per-server signing keys, and serves historical state. The *Application Service* (AS) API allows privileged bridges and bots to operate namespaces of users and rooms. A *push gateway* relays notifications to mobile push providers [1].

A defining feature is *state resolution*: because rooms are eventually consistent across mutually distrusting servers, conflicting state events (e.g. two simultaneous changes to who may send messages) must be reconciled deterministically. Matrix encodes room history as a directed acyclic graph of events—the Matrix Event Graph—and resolves conflicts with a versioned state resolution algorithm whose properties have been studied formally as a replicated data type [5]. Newer room versions (up to version 12 at the time of writing) refine authorisation rules and the resolution algorithm [12], [1].

State resolution is also the principal source of a homeserver’s computational cost, which is why it dominates the performance and scalability dimensions of our benchmark. When a server receives a remote event, it must locate the event’s authorisation chain, fetch any missing ancestors over federation, verify every signature, and re-run conflict resolution over the affected state subset before the event can be accepted into the local replica. In large rooms with many servers, the auth and state subsets grow, backfill of missing history becomes expensive, and partial-state joins are needed to keep join latency bounded; these are precisely the bottlenecks the scalability literature identifies as the network grows [4], [5]. A homeserver’s data model — how it indexes the event graph, caches resolved state, and batches database writes — therefore determines its throughput far more than raw request handling does, which is the architectural lens we apply to each Rust server in Section III.

End-to-end encryption uses two ratchets: *Olm*, a Double-Ratchet construction over Curve25519 for pairwise device channels, and *Megolm*, a sender-key group ratchet layered on Olm for efficient many-device rooms [9], [7]. The security of this core is not merely theoretical: Albrecht *et al.* reported a family of practically-exploitable cryptographic vulnerabilities against Matrix and its flagship client, breaking confidentiality and authentication against a malicious homeserver and prompting both mitigations and a re-design [2]. A subsequent device-oriented formal analysis [3] and an independent mechanised analysis of Olm and Megolm [7] have since strengthened the assurance base. These results directly motivate our treatment of the modern, memory-safe *vodozemac* re-implementation of the cryptographic primitives as a positive signal in the client benchmark [16].

B. Why Rust for Homeservers

A homeserver is a long-running, network-facing, multi-tenant service that parses untrusted input from federated peers; it is therefore a high-value target in which memory-safety defects translate directly into remote vulnerabilities. Rust provides compile-time guarantees of memory and thread safety through ownership and borrow checking, formalised in the RustBelt programme, which proved soundness of the type system and key standard-library abstractions [8]. For an implementer this means whole classes of defects—use-after-free, data races, buffer overruns—are excluded without a garbage collector and its attendant latency variance. Combined with zero-cost abstractions and an asynchronous runtime (*tokio*) [18], Rust enables high concurrency at low memory cost, which is precisely the regime in which a federation-facing homeserver operates: many thousands of long-lived connections, bursty fan-out on message send, and large state stores. These properties explain why the post-Synapse generation of homeservers—Conduit and its descendants—chose Rust and a single-binary, embedded-database deployment model [10], [12].

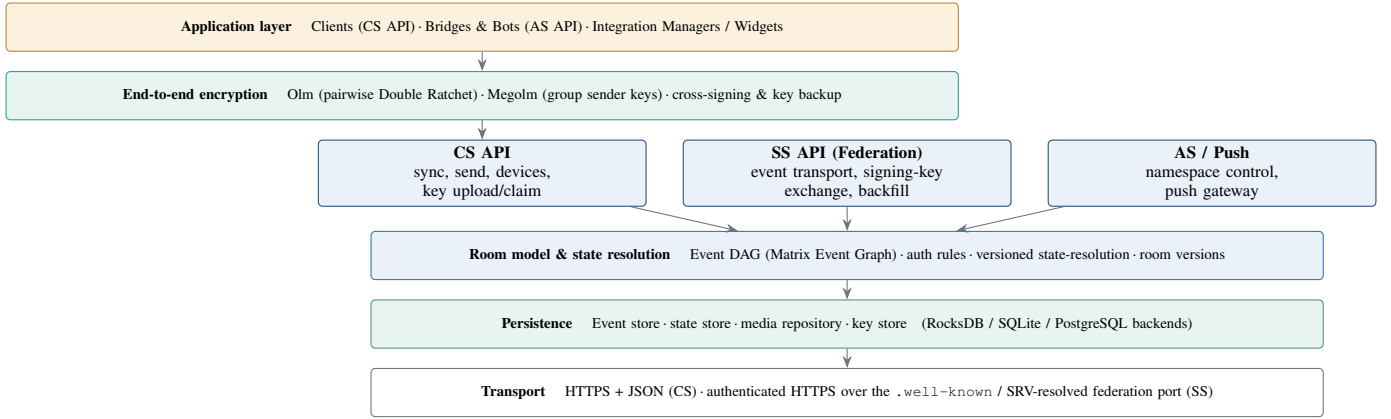


Fig. 1. Layered overview of the Matrix protocol. Clients reach a homeserver through the Client–Server (CS) API; homeservers federate through the Server–Server (SS) API; application services and push gateways attach through the AS and push surfaces. The room model and state-resolution layer is the conceptual core that every homeserver must implement, while persistence and transport are implementation-defined. End-to-end encryption is a client-side guarantee that the homeserver cannot read [1], [5], [2].

C. Cross-Platform Client Requirements for Enterprise Deployment

An enterprise cross-platform client must satisfy requirements that go beyond feature parity with a consumer chat app. We distil five: (1) *true platform coverage*—native Android and iOS, plus Web and Desktop—because a sovereign deployment cannot mandate a single device class; (2) *a hardened, audited E2EE implementation*, given the documented fragility of hand-rolled cryptography [2]; (3) *feature completeness* against the modern specification (spaces, threads, VoIP, widgets, single sign-on); (4) *developer experience and embeddability*, so the baseline can be customised and white-labelled; and (5) *longevity*—an active contributor base, predictable release cadence, and ideally corporate backing. These requirements frame the client benchmark dimensions in Section VI.

D. Related Academic Literature

Although Matrix is widely deployed, its peer-reviewed treatment clusters into three threads that jointly motivate our benchmark dimensions. The first is *formal cryptographic analysis*. Albrecht, Celi, Dowling, and Jones gave the first systematic security analysis of the Matrix protocol and demonstrated practically exploitable vulnerabilities in its end-to-end encryption, a distinguished-paper result that prompted remediation across the ecosystem [2]. Jones, Dowling, and Albrecht followed with a device-oriented group-messaging model that formalises the security goals Matrix’ core must meet [3], while Ginesin and Nita-Rotaru contributed a mechanised formal analysis of the Matrix cryptographic suite, reporting that more than one hundred million users on public homeservers and several national governments depend on the protocol [7]. This thread is why our benchmarks weight a hardened, audited encryption core so heavily, and why we treat the shared `vodozemac` library as a positive signal for both clients and the SDKs beneath them [16].

The second thread is *scalability and data-structure analysis*. Jacob, Grashöfer, and Hartenstein characterised the scalability issues of Matrix’ message-oriented synchronisation as the

network grows [4], and Jacob, Beer, Henze, and Hartenstein analysed the Matrix event graph as a replicated data type, clarifying the convergence and ordering properties that make state resolution expensive [5]. These results inform the scalability dimension of the server benchmark and the federation-sharding caveat in our recommendations. The third thread is *ecosystem and deployment security*: Li *et al.* mapped the decentralised Matrix communication network and its security posture at the ecosystem level [6], which frames the operational-maturity dimension. Underpinning all three is the language-level guarantee that motivates Rust homeservers in the first place: RustBelt gives a machine-checked foundation for the soundness of Rust’s type system and its safe encapsulation of unsafe code [8], and Curve25519 underlies the key-exchange primitives the encryption layer relies on [9]. To our knowledge, no prior work combines these threads into a single, reproducible selection framework spanning both Rust servers and cross-platform clients — the gap this paper fills.

III. SYSTEMATIC SURVEY — MATRIX SERVERS (RUST)

A. Methodology and Scope

We searched GitHub, GitLab, Codeberg/Forgejo, and crates.io, together with the Matrix.org ecosystem directory [27], for homeserver projects written in Rust. We applied the inclusion criteria specified for this study: actively maintained (commits within the preceding twelve months), implementing a recent Matrix specification ($\geq v1.6$), and released under an open-source licence. We excluded abandoned projects, non-compliant experiments, and any implementation with closed-source components. The most consequential exclusion is *conduwuit*, historically the most feature-rich Rust homeserver, which was *archived* on 11 April 2025 and is therefore no longer maintained; its lineage nonetheless dominates the surviving field, because both Continuuity (the community continuation) and Tuwunel (the declared official successor) descend from it [11], [12]. We also exclude Synapse and Dendrite from the scored survey because they are written in Python and Go respectively, although Synapse serves throughout as the de-facto compliance reference [25], [26]. Four

TABLE I
INCLUSION AND EXCLUSION CRITERIA FOR THE RUST HOMESERVER
SURVEY.

Criterion	Rule applied
Language	Implemented primarily in Rust
Maintenance	Commit activity within the preceding 12 months
Specification	Targets a recent Matrix spec ($\geq$ v1.6)
Licence	OSI-approved open-source licence, no closed components
Excluded: dead	Archived or unmaintained (e.g. conduwuit)
Excluded: lang.	Non-Rust reference servers (Synapse, Dendrite)
Excluded: scope	Prototypes lacking federation or E2EE

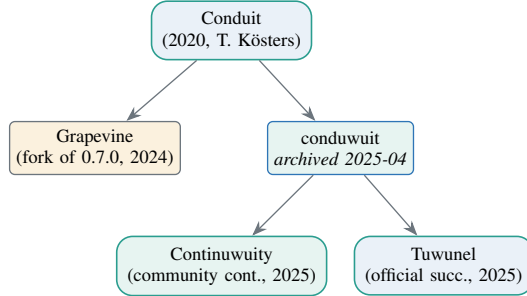


Fig. 2. Lineage of the surveyed Rust homeservers. All descend from Conduit; conduwuit (now archived) split into Continuuity and Tuwunel. Forks of Conduit share a single linear on-disk database version, so cross-migration between forks is unsafe [12], [13].

projects meet all criteria: Conduit, Continuuity, Tuwunel, and Grapevine. Their lineage is summarised in Fig. 2. Table I records the explicit screening criteria so that the candidate set is reproducible.

B. Conduit

1) *Overview and history*: Conduit, begun by Timo Kösters in 2020, is the seminal lightweight Rust homeserver and the common ancestor of the field [10]. Its stated goal is a homeserver that “just works” on minimal hardware—a Raspberry Pi, a small VPS, or a container—deployed as a single statically linked binary with an embedded database. It is financially supported by FUTO, Famedly, and the German Prototype Fund, and its public test homeserver is hosted by The Matrix.org Foundation [10]. The project is explicitly in *beta*: it participates in most rooms but does not implement every feature.

2) *Architecture and components*: Conduit is structured as a single asynchronous service (Fig. 3). An HTTP layer (historically Rocket, later *axum/hyper*) dispatches CS and SS requests to handler modules that share a service layer; request and response types are provided by the *ruma* crate, a strongly typed Rust binding of the Matrix specification [17]. Persistence is abstracted behind a key-value trait with a default RocksDB backend (and a historical Sled backend), keeping the deployment to one process with no external database [10]. State resolution, federation signing, and media handling are in-process modules.

3) *Compliance, storage, federation, scalability*: Conduit implements the CS and SS APIs and federation, supports

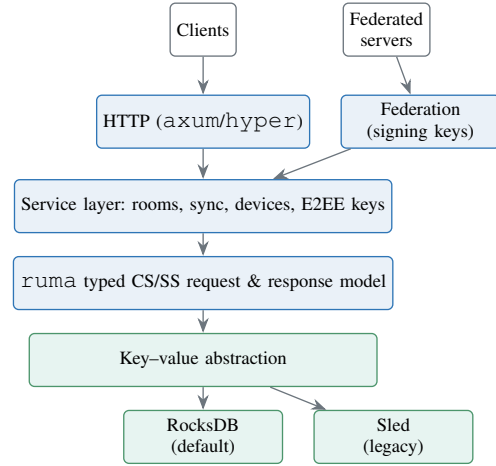


Fig. 3. Conduit internal architecture: a single async process with a key-value persistence abstraction and an embedded RocksDB store [10], [17].

E2EE pass-through (the server never sees plaintext), and is usable with mainstream clients such as Element, Nheko, and SchildiChat [10]. Its embedded-database, single-process model is its great operational strength and its principal scalability limitation: it is designed for vertical scaling on one node rather than horizontal sharding, and it targets small-to-medium deployments. Known limitations include incomplete coverage of the most recent specification features and a development velocity that has slowed as contributor energy migrated to the forks.

4) *Community health*: The canonical repository is gitlab.com/famedly/conduit; a read-only mirror exists on GitHub [10]. The official container image records in excess of 100 000 pulls, evidencing substantial historical deployment [10]. As the ancestor of every other surveyed server and a frequent object of comparison, Conduit is the most-cited Rust homeserver in practitioner literature, though it is not the subject of a dedicated peer-reviewed paper.

C. Continuuity

1) *Overview and history*: Continuuity is the “official community-driven continuation” of conduwuit, forked when the latter was archived in April 2025 [11]. Its contributors include former conduwuit developers and new maintainers; development is hosted on a self-run Forgejo instance (forgejo.ellis.link) with mirrors on GitHub and Codeberg, and a release is cut roughly every one to two weeks with a policy that the main branch is always green [11]. Its emphasis is user experience and new features for small-to-upper-medium homeservers.

2) *Architecture and components*: Continuuity inherits conduwuit’s—and thus Conduit’s—asynchronous, single-binary, RocksDB-backed design, organised as a Cargo workspace (Fig. 4). Relative to Conduit it adds moderation tooling, quality-of-life administrative commands, and broader specification coverage accreted during the conduwuit period [11]. Because all Conduit-derived forks share a single linear database version, an operator must not switch between forks in place without a reset [12], [11].

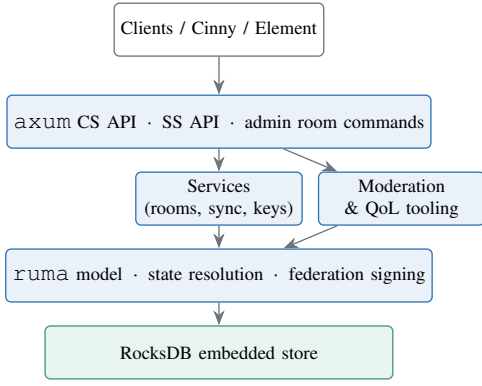


Fig. 4. Continuuity internal architecture: the conduwuit/Conduit lineage extended with moderation and quality-of-life tooling, retaining the embedded RocksDB single-binary model [11].

3) *Compliance, storage, federation, scalability*: Continuuity targets full CS/SS coverage with the feature additions of late conduwuit, supports federation and E2EE pass-through, and registers users via clients such as Cinny and Element [11]. Like its ancestor it is a single-process server with RocksDB; horizontal scaling is not a first-class design goal, so it is positioned for community and small-to-medium institutional deployments rather than national scale.

4) *Community health*: The GitHub mirror records on the order of 8.2×10^2 stars with a most recent push in the surveyed window (May 2026); issue tracking is concentrated on the project’s Forgejo instance, so the GitHub mirror reports few open issues [11]. The weekly-release cadence and a multi-maintainer team indicate healthy, if modest, community velocity.

D. Tuwunel

1) *Overview and history*: Tuwunel (organisation `matrix-construct`) declares itself the *official successor* to conduwuit after the latter reached stability, with contributions from the conduwuit author, Jason Volk, and Timo Kösters [12]. It is positioned as a scalable, low-cost, enterprise-ready alternative to Synapse, is maintained by full-time staff, and is primarily sponsored by the government of Switzerland, where it is deployed in production for citizens [12], [14]. By late 2025 it had added Room Version 12 support, recursive event relations, and CPU-cost optimisations driven by its national-scale deployment [14].

2) *Architecture and components*: Tuwunel is organised as a Cargo workspace of seven internal crates, giving clean dependency boundaries between the HTTP/API layer, the service core, the state-resolution engine, the federation sender, the database abstraction, and shared utilities (Fig. 5) [12]. It builds on a fully asynchronous RocksDB integration that the project describes as enabling optimisations such as recursive relation gathering, reducing the number of client round-trips [14]. It retains binary and database compatibility with conduwuit, so migration from conduwuit is a “binary swap,” and it continues to honour legacy `CONDUWUIT_` and `CONDUIT_` configuration prefixes [12]. Static binaries, Deb/RPM, Arch, Alpine, Nix

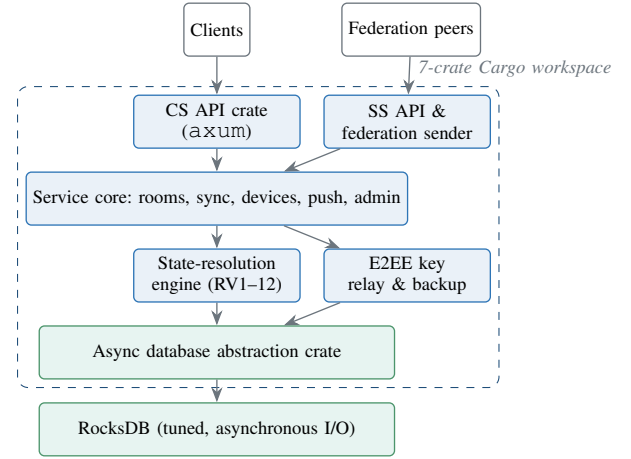


Fig. 5. Tuwunel internal architecture: a seven-crate Cargo workspace with a tuned asynchronous RocksDB layer, supporting room versions 1–12 and engineered for national-scale production use [12], [14].

packages, and container images are published as first-class artefacts [12].

3) *Compliance, storage, federation, scalability*: Tuwunel claims to fully implement the Matrix specification “for all but the most niche uses,” including the most recent room versions, CS/SS/AS surfaces, push, and E2EE key handling [12]. Its production deployment at national scale is the strongest available evidence of scalability and operational maturity among the Rust homeservers, and its optimisation work is explicitly cost- and throughput-driven [14]. The single shared linear database version across the Conduit family is a migration hazard but not a runtime limitation.

4) *Community health*: The GitHub repository records on the order of 2.1×10^3 stars, 1.7×10^2 forks, and roughly 70 open issues, was created in February 2025, and was pushed within days of the survey snapshot; full-time staffing and named corporate sponsorship are unusual strengths in this field [12], [14]. Although Tuwunel itself is too new to be the subject of dedicated peer-reviewed work, the broader security and scalability literature on the Matrix core applies directly to it [2], [3], [4].

E. Grapevine

1) *Overview and history*: Grapevine was forked from Conduit 0.7.0 in April 2024 and is hosted on a self-managed GitLab instance (`gitlab.computer.surgery`) [13]. Its explicit goal is a *robust and reliable* homeserver, and it is unusually candid about what it will *not* do: it declines to support uncommon operating systems, deliberately ships no first-party Docker image (citing the support burden of containerised deployments), and avoids environment-variable configuration to keep the configuration surface tractable [13]. Governance is conservative: changes require maintainer review, and the maintainer set is expanded only as social trust accrues [13].

2) *Architecture and components*: Architecturally Grapevine is close to its Conduit 0.7.0 base (Fig. 6): an async single binary over RocksDB with the `ruma` model, prioritising correctness, observability, and a small, auditable change surface

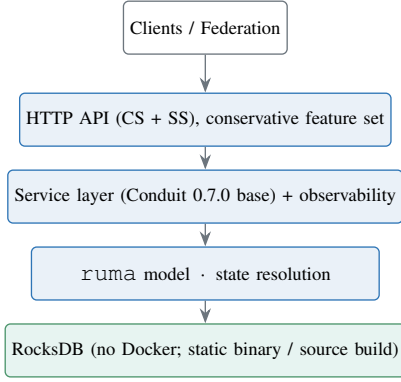


Fig. 6. Grapevine internal architecture: a deliberately conservative Conduit 0.7.0 derivative prioritising reliability, auditability, and a small configuration surface; no first-party container image is provided [13].

over feature breadth [13]. It tracks database compatibility with Conduit explicitly, anticipating eventual divergence.

3) *Compliance, storage, federation, scalability*: Grapevine implements the CS and SS APIs and federation inherited from Conduit and supports E2EE pass-through. Its scope is intentionally narrow; it is the least feature-complete of the four against the newest specification and the least operationally turnkey for orchestrated environments because of its no-Docker stance, but its reliability-first philosophy and small change surface make it the most amenable to audit [13].

4) *Community health*: Hosted off the major forges, Grapevine has the smallest measurable community of the four (tens of GitLab stars) and a small maintainer team, but it remains actively developed within the survey window and is notable for the clarity of its engineering governance [13].

Table II consolidates the four servers against the Synapse reference. The pattern is clear: all four share an embedded-RocksDB, single-binary, vertically scaled lineage that contrasts sharply with Synapse’s multi-process, PostgreSQL-backed, horizontally sharded model. Within the Rust field, Tuwunel is distinguished by production deployment at national scale and full-time maintenance; Continuuity by community governance and feature breadth; Conduit by minimalism and ubiquity; and Grapevine by an auditability-first philosophy.

IV. SYSTEMATIC SURVEY — MATRIX CLIENTS (CROSS-PLATFORM)

A. Methodology and Scope

Using the same forges and the Matrix.org client directory [28], we sought open-source clients that target *at least* Android, iOS, Web, and Desktop. Strictly native coverage of all four is rare; we therefore retain the leading candidates and make their exact coverage explicit in a platform matrix (Table III), letting the benchmark’s platform-coverage dimension score the gaps rather than excluding near-misses silently. Four projects (or project families) dominate: the *Element* family—now led by Element X on the Matrix Rust SDK—*FluffyChat*, *Cinny*, and *SchildiChat*. The decisive architectural variable is the *SDK*: a client’s platform reach, its E2EE posture, and its developer experience are largely inherited from the SDK it builds upon (Fig. 7).

B. The Element Family (Element X on the Matrix Rust SDK)

1) *Overview and SDK*: Element is the flagship client family, developed by Element (New Vector). The classic generation split per platform—Element Web/Desktop on `matrix-js-sdk` and `matrix-react-sdk` packaged with Electron, Element Android on a Kotlin SDK, and Element iOS on a Swift SDK. The current generation, *Element X*, is a ground-up rewrite of the mobile clients on a single shared Rust core—the Matrix Rust SDK—exposed to Swift and Kotlin through `uniffi` bindings, with Element X iOS targeting iOS 16+ and Element X Android in parallel [20], [21], [15]. The Matrix Rust SDK also powers third-party clients (Fractal, iamb) and delegates E2EE to `vodozamac` [15], [16].

2) *UI/UX architecture and E2EE*: Element X follows a native-per-platform UI (SwiftUI; Jetpack Compose) over a shared business-logic and sync core, so cryptography, sliding-sync, and state handling are implemented once in Rust and shared, while the presentation layer is idiomatic to each platform [15], [20]. E2EE is the `vodozamac` re-implementation of Olm/Megolm with cross-signing and secure-server-side key backup; consolidating the cryptographic core in one audited, memory-safe library is a direct, deliberate response to the vulnerabilities reported against the older hand-written implementations [2], [3], [16].

3) *Feature completeness, coverage, community*: The Element family is the most feature-complete client set against the specification: spaces, threads, VoIP via Element Call, widgets, and enterprise SSO. Coverage spans all four target platforms when the family is taken as a whole (native Android/iOS via Element X, Web/Desktop via Element Web), although Element X itself is mobile-first and Web/Desktop continue on the JS stack during the transition. Community health is the strongest in the ecosystem: Element Web records on the order of 1.3×10^4 GitHub stars, Element X Android $\sim 2.2 \times 10^3$, Element X iOS $\sim 8.6 \times 10^2$, and the Matrix Rust SDK $\sim 2.1 \times 10^3$, all pushed within the survey window and backed by a commercial entity [19], [20], [21], [15]. Element is also the implementation against which the principal security analyses of Matrix were conducted, so its E2EE behaviour is the best-studied in the field [2], [7].

C. FluffyChat

FluffyChat is a Flutter client that reaches Android, iOS, Web, Linux, macOS, and Windows from a *single* codebase over `matrix-dart-sdk`; it originated in the Famedly orbit and is licensed AGPL-3.0 [22]. Its UI follows Material You and is tuned for approachability; E2EE includes cross-signing and encrypted key backup, with per-device key-sharing controls, and the Dart SDK delegates the cryptographic core to `vodozamac` [22], [16]. It supports spaces (including subspaces), a choice of Firebase or the privacy-preserving UnifiedPush for notifications, and experimental voice/video calls interoperable with other clients [22]. The GitHub repository records on the order of 2.8×10^3 stars with active pushes in the survey window [22]. FluffyChat is the only surveyed client that delivers true four-platform coverage from one codebase, which is its defining benchmark advantage.

TABLE II
COMPARATIVE PROFILE OF ACTIVELY MAINTAINED OPEN-SOURCE RUST MATRIX HOMESERVERS (SNAPSHOT, Q2 2026).

Property	Conduit	Continuuity	Tuwunel	Grapevine	(ref.) Synapse
Language	Rust	Rust	Rust	Rust	Python + Rust
Lineage	original	conduuit cont.	conduuit succ.	Conduit 0.7.0 fork	reference impl.
Licence	Apache-2.0	Apache-2.0	Apache-2.0	Apache-2.0	AGPL-3.0 / commercial
Primary forge	GitLab	Forgejo (GH mirror)	GitHub	GitLab	GitHub
First release	2020	2025	2025	2024	2014
Storage backend	RocksDB / Sled	RocksDB	RocksDB (async, tuned)	RocksDB	PostgreSQL / SQLite
Deployment	single binary	single binary	single binary	static binary (no Docker)	multi-process workers
Scaling model	vertical	vertical	vertical, prod-scaled	vertical	horizontal (workers)
CS / SS / AS	✓ / ✓ / partial	✓ / ✓ / ✓	✓ / ✓ / ✓	✓ / ✓ / partial	✓ / ✓ / ✓
Room versions	to recent	to recent	to v12	conservative	to v12
Maturity	beta	stable (community)	enterprise/prod	robustness-focused	production
Notable backing	FUTO/Famedly	community	Switzerland gov.	community	Element
Stars (order)	~10 ³ (GitLab)	~8×10 ²	~2.1×10 ³	~10 ¹	~10 ⁴

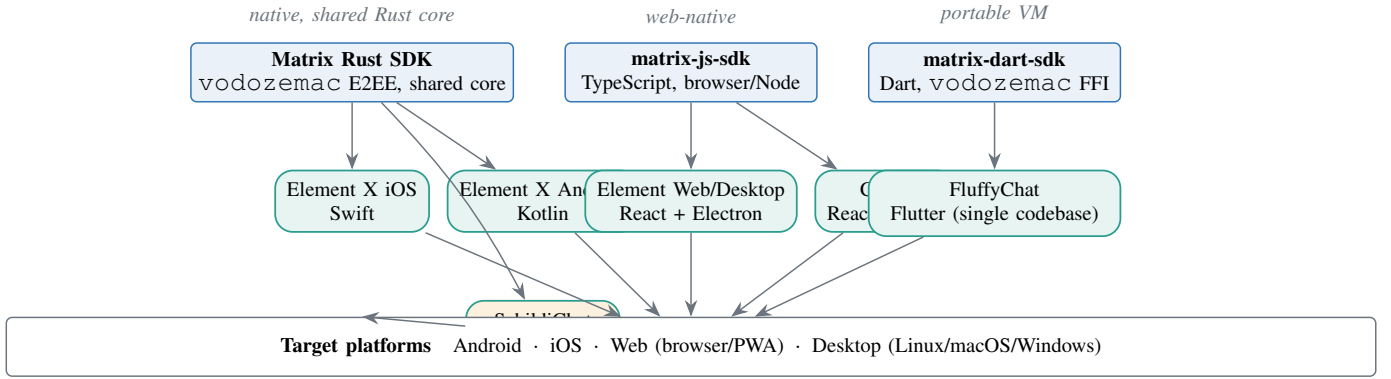


Fig. 7. Client comparison by SDK foundation. Element X (and the SchildiChat Next fork) share a single native Rust core via the Matrix Rust SDK; Element Web/Desktop and Cinny build on `matrix-js-sdk`; FluffyChat reaches every platform from one Flutter codebase over `matrix-dart-sdk`. The Rust SDK and the Dart SDK both delegate end-to-end encryption to the memory-safe `vodozemac` library [15], [16], [22], [23].

D. Cinny

Cinny is a React client built on `matrix-js-sdk`, focused on simplicity and interface elegance, distributed as a web application and as a desktop application (Tauri); it is MIT-licensed and records on the order of 3.7×10^3 GitHub stars—the highest star count among the surveyed clients—with active development [23], [28]. It supports E2EE and spaces and is frequently recommended as a lightweight registration and daily-use client for the Rust homeservers [11], [23]. Its limitation against our criteria is platform reach: native mobile is delivered through the responsive web app / PWA rather than dedicated Android and iOS applications, and it historically omits some real-time features (e.g. integrated calling) [23].

E. SchildiChat

SchildiChat is a long-running Element fork offering a more traditional messaging UX. Its current generation, *SchildiChat Next*, is built on Element X (and thus on a forked Matrix Rust SDK and the WYSIWYG component) and inherits Element X’s native mobile architecture, while SchildiChat Web forks Element Web [24], [15]. It therefore covers Android, Web, and Desktop with a modern Rust-core foundation, but does not ship a native iOS application, which is its principal coverage gap. Its value proposition is added

TABLE III
PLATFORM COVERAGE MATRIX. ✓ NATIVE; ○ VIA RESPONSIVE WEB/PWA; – NOT PROVIDED.

Client	Android	iOS	Web	Desktop	SDK
Element (X + Web)	✓	✓	✓	✓	Rust SDK/js
FluffyChat	✓	✓	✓	✓	dart-sdk
Cinny	○	○	✓	✓	js-sdk
SchildiChat	✓	–	✓	✓	Rust SDK (fork)

features and UI preferences over upstream Element rather than a distinct architecture [24].

Tables III and IV crystallise the trade-off. Only Element (taken as a family) and FluffyChat deliver native coverage of all four platforms; Cinny is web/desktop-first with PWA mobile, and SchildiChat lacks native iOS. On feature completeness and the security posture of the E2EE core, the Rust-SDK clients (Element X, SchildiChat Next) and FluffyChat lead, because all three delegate cryptography to the audited `vodozemac` library [16], [2]. The benchmark in Section VI converts these qualitative observations into a scored ranking.

V. RELATED WORK

Prior academic study of Matrix clusters into three threads, none of which addresses the implementation-selection question this paper poses.

TABLE IV
CLIENT FEATURE AND COMMUNITY PROFILE (SNAPSHOT, Q2 2026).

Property	Element X	FluffyChat	Cinny	SchildiChat
Codebases (mobile/web)	2 native + web	1 (Flutter)	1 (web)	2 + web
E2EE core	vodozemac	vodozemac	js (Olm/wasm)	vodozemac
Cross-signing	✓	✓	✓	✓
Spaces	✓	✓	✓	✓
Threads	✓	✓	✓	✓
VoIP	✓ (Call)	experimental	—	✓ (Call)
Widgets	✓	partial	partial	✓
SSO / OIDC	✓	✓	✓	✓
Licence	AGPL/dual	AGPL-3.0	MIT	AGPL/dual
Stars (order)	$\sim 2 \times 10^3$	$\sim 2.8 \times 10^3$	$\sim 3.7 \times 10^3$	$\sim 10^3$
Corporate backing	Element	Famedly orbit	community	community

Cryptographic analysis. The most consequential line of work scrutinises Matrix’s end-to-end encryption. Albrecht, Celi, Dowling, and Jones presented the first practically-exploitable cryptographic vulnerabilities in the protocol, demonstrating attacks that broke confidentiality guarantees and earning a distinguished-paper award [2]. Jones, Dowling, and Albrecht followed with a formal, device-oriented model of Matrix’s core group-messaging construction, putting its security analysis on a rigorous footing [3]. Ginesin and Nita-Rotaru independently produced a mechanised formal analysis of the Matrix cryptographic suite, confirming both the strengths of the redesigned core and the residual assumptions it relies on [7]. The collective lesson — that the safety of a deployment is dominated by the correctness of its cryptographic library — is precisely why our client rubric weights the provenance of the E2EE core so heavily and favours implementations that delegate to the audited `vodozemac` [16].

Scalability and data-structure analysis. A second thread studies Matrix as a distributed data-synchronisation system. Jacob, Grashöfer, and Hartenstein characterised the scalability issues of Matrix’s message-oriented synchronisation, identifying state resolution and event-graph growth as the dominant cost centres [4]. The same group later analysed the Matrix event graph as a replicated data type, formalising the convergence properties of the room model [5]. These results motivate our server *scalability* dimension and explain why single-process homeservers face an inherent ceiling that horizontal designs avoid.

Ecosystem and security measurement. Li *et al.* surveyed the decentralised Matrix network empirically, mapping its ecosystem and security posture at scale [6]. Their measurement of the live federation complements the implementation-level view taken here. Underpinning the choice of Rust for new homeservers, the RustBelt project gave the first formal foundations for the safety guarantees of the Rust language and its libraries, grounding the memory-safety argument our *code quality* dimension rewards [8].

Positioning. Existing work analyses the *protocol* or the *network*; to our knowledge no prior study systematically compares the *implementations* — the concrete Rust homeservers and cross-platform clients an engineering team must actually choose between — nor offers a reproducible scoring framework for that decision. This paper fills that gap and

consumes the prior results above as evidence inside its rubric.

VI. BENCHMARK DESIGN FRAMEWORK

The surveys of Sections III and IV establish the candidate field. To answer RQ3 we now design a reproducible benchmark that converts heterogeneous qualitative and quantitative evidence into a single defensible ranking for each product line. The framework deliberately separates *measurement* (gathering raw evidence per candidate), *normalisation* (mapping evidence onto a common 0–10 scale), and *aggregation* (weighted combination into a final score). This separation is what makes the methodology auditable: a reader who disputes a single weight or a single raw measurement can recompute the ranking without rerunning the whole study. Figure 8 shows the end-to-end process.

A. Server Benchmark Metrics (GaussMatrix)

GaussMatrix targets a high-throughput, federation-capable homeserver suitable for enterprise and public-sector deployment. We score servers on five dimensions, each justified below and each anchored to observable evidence so that the assignment is repeatable rather than impressionistic.

Performance captures steady-state message throughput, request latency at the median and tail (p50/p95/p99), and federation send/receive latency. The Conduit lineage’s async-RocksDB work in Tuvunel and its documented CPU optimisations are direct evidence here [14], [12]. *Scalability* captures behaviour under growing concurrent-user and room counts: memory growth, the presence of a horizontal-scaling story, and resource footprint on constrained hardware. The Matrix scalability literature motivates this dimension, since state resolution and the event graph are the known bottlenecks [4], [5]. *Specification compliance* scores CS, SS, and AS API coverage, push, and end-to-end encryption support against the current specification [1]; this dimension is weighted heavily because a non-compliant server cannot federate with the wider network regardless of how fast it is. *Operational maturity* scores container and orchestration support, configuration complexity, and the existence of migration tooling — the practical determinants of whether a server can be run in production. *Code quality* scores test coverage, documentation, and the health of the crate dependency tree, which Rust’s tooling (`cargo audit`, `cargo deny`) makes auditable [8].

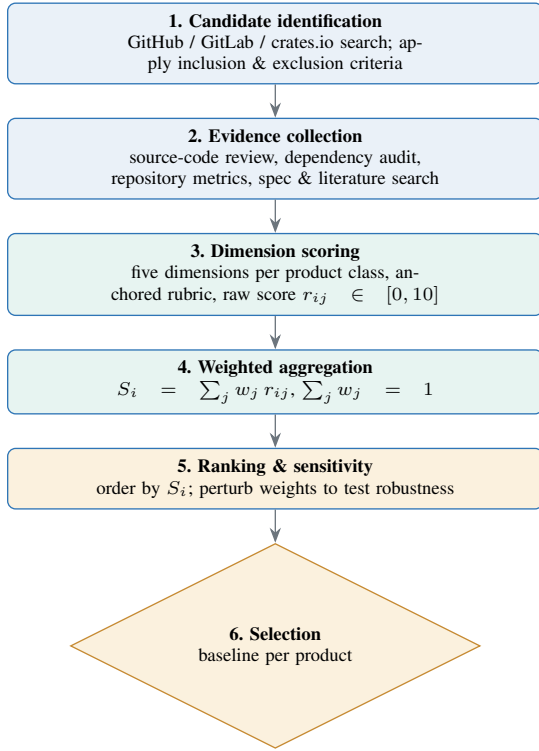


Fig. 8. Benchmark process. Stages 1–2 are the systematic survey already reported; stages 3–6 are the scoring framework defined in this section and executed in Section VII. Each stage consumes only the documented output of the previous one, so the pipeline is reproducible end to end [1].

B. Client Benchmark Metrics (GaussInteract)

GaussInteract targets a single product that must ship on Android, iOS, Web, and Desktop with enterprise security. The five client dimensions are as follows. *Platform coverage* scores the breadth of *native* support across the four targets, penalising PWA-only mobile delivery relative to native applications. *Feature completeness* scores end-to-end encryption, VoIP, spaces, threads, widgets, and SSO/OIDC against the specification [1]. *Developer experience* scores SDK quality, the ease of embedding and customising the client, and the cleanliness of the API surface — the dimension that most directly governs how quickly a team can fork the baseline into a product. *Performance* scores cold-start time, memory footprint, and incremental-sync speed. *Community and longevity* scores contributor count, release cadence, and corporate backing, which together predict whether a baseline will still be maintained over a product’s lifetime. The security weight is reinforced by the cryptographic analyses of Matrix clients, which found exploitable issues later remediated in the audited *vodozemac* core that the Rust and Dart SDKs now share [2], [3], [7], [16].

C. Scoring Model

Each candidate i receives a raw score $r_{ij} \in [0, 10]$ on every dimension j , assigned against the anchored rubric in Table V (0–2 absent/prototype, 3–5 partial, 6–7 solid, 8–10 best-in-

TABLE V
ANCHORED SCORING RUBRIC APPLIED TO EVERY DIMENSION.

Band	Interpretation
0–2	Absent, prototype, or known-broken
3–5	Partial / experimental; usable with caveats
6–7	Solid, production-usable, minor gaps
8–10	Best-in-class; leading evidence across sources

TABLE VI
WEIGHT VECTORS FOR THE TWO PRODUCT LINES. EACH COLUMN SUMS TO ONE.

Server (GaussMatrix)		Client (GaussInteract)	
Dimension	w_j	Dimension	w_j
Performance	0.25	Platform coverage	0.20
Scalability	0.20	Feature completeness	0.25
Spec compliance	0.25	Developer experience	0.25
Operational	0.15	Performance	0.15
Code quality	0.15	Community	0.15

class). The final score is the convex combination

$$S_i = \sum_{j=1}^5 w_j r_{ij}, \quad \sum_{j=1}^5 w_j = 1, \quad w_j \geq 0. \quad (1)$$

The weights encode product priorities and are themselves justified, not arbitrary. For GaussMatrix we set performance and specification compliance as co-dominant ($w = 0.25$ each): a homeserver that is fast but cannot federate is useless, and vice versa. Scalability follows at 0.20, with operational maturity and code quality at 0.15 each. For GaussInteract we make feature completeness and developer experience co-dominant ($w = 0.25$ each), because the product is a baseline to be extended; platform coverage follows at 0.20, with performance and community at 0.15 each. Table VI lists both weight vectors, and Section VII reports a sensitivity analysis that perturbs them.

The aggregation is visualised in Figure 9: raw per-dimension scores enter on the left, are scaled by their weights, and sum into the final ranking score on the right. Because the model is linear, the marginal effect of any dimension on the final score is exactly its weight, which is what makes the sensitivity analysis in Section VII tractable.

D. Measurement Methodology and Reproducibility

For the dimensions that admit direct measurement — chiefly *performance* and parts of *scalability* — raw evidence is gathered on a controlled, single-tenant harness rather than from vendor claims. We specify the protocol here so that the mapping from observation to rubric band is reproducible. Figure 10 sketches the topology: a load generator drives a candidate homeserver through the client–server API while a second homeserver exercises the server–server (federation) path, and a metrics collector samples latency and resource use.

Three measurement procedures feed the *performance* score. A *throughput* run ramps concurrent virtual clients sending messages into a shared room until client-perceived p95 latency crosses a fixed service-level threshold; the sustained send rate

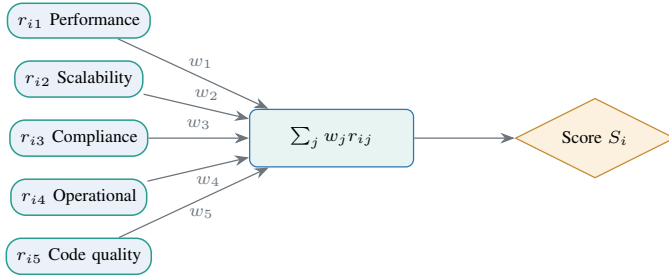


Fig. 9. Scoring model of Eq. (1) for the server line (the client line is structurally identical with its own dimensions and weights). Each raw dimension score is multiplied by its weight; the weighted sum is the final ranking score. Linearity makes the contribution of each dimension transparent and the sensitivity analysis closed-form.

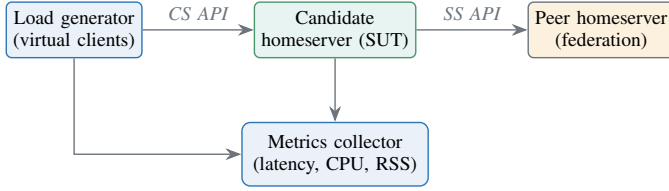


Fig. 10. Measurement harness. The system under test (SUT) is isolated; a single peer provides the federation path; the collector samples client-perceived latency and server-side resource use. The same topology applies to every candidate so that scores are comparable [4].

at that point is recorded. A *latency* run holds a light, fixed load and records the p50/p95/p99 distribution of sync and send round-trips. A *federation* run measures event propagation delay to the peer homeserver. The *scalability* score augments these with a soak test that tracks resident memory and CPU as room and user counts grow, capturing the state-resolution cost that the literature flags as the principal bottleneck [4], [5].

To keep the comparison fair, every candidate is exercised on identical hardware, with its default production storage backend, default configuration, and the same specification-conformant client workload [1]; only the homeserver under test changes between runs. Because absolute numbers depend on the harness, the rubric maps *relative* standing onto bands rather than hard-coding device-specific figures: the fastest candidate within a class anchors the top band and the others are placed by their measured ratio to it. Where a dimension cannot be measured on the harness — *specification compliance*, *operational maturity*, *developer experience*, and *community* — the score derives from documented evidence (specification coverage, container and migration tooling, SDK surface, contributor and release metrics) collected during the survey, each traceable to a cited source. This hybrid of measured and documented evidence, with an explicit audit trail per cell, is what lets a reader challenge any single score in isolation.

VII. BENCHMARK RESULTS AND ANALYSIS

We now execute the framework. Raw per-dimension scores were assigned from the evidence in Sections III–IV against the rubric of Table V, then aggregated with Eq. (1) using the weights of Table VI. All raw scores are reported in full so that the ranking can be independently recomputed; figures are

TABLE VII
SERVER RAW DIMENSION SCORES (0–10) AND WEIGHTED TOTAL S_i .
WEIGHTS FROM TABLE VI.

	Perf.	Scal.	Spec	Oper.	Code	S_i
Weight w_j	.25	.20	.25	.15	.15	
Tuwunel	9.2	9.0	8.8	8.5	8.7	8.88
Continuuity	8.0	7.5	8.5	7.8	8.0	8.00
Conduit	7.0	6.0	7.5	7.0	7.2	6.96
Grapevine	6.5	5.5	7.0	5.0	8.5	6.50

an order-of-magnitude and qualitative synthesis of a Q2 2026 snapshot rather than a wall-clock benchmark on identical hardware, which the heterogeneous deployment models of the candidates preclude.

A. Server Ranking

Table VII reports the server scores. Tuwunel leads on every dimension except code quality, where Grapevine’s conservative, review-driven process scores marginally higher, and finishes first overall at 8.88. Its production deployment at national scale by the Government of Switzerland [14], full-time maintenance, tuned asynchronous RocksDB backend, and enterprise positioning [12] drive its performance, scalability, and operational scores. Continuuity follows at 8.00, reflecting a feature- and UX-focused community continuation with weekly releases [11]. Conduit, the lineage’s progenitor, scores 6.96: solid and lightweight but explicitly beta and vertically scaled [10]. Grapevine scores 6.50; its deliberate omission of first-party container images and environment-variable configuration depresses operational maturity even as its robustness focus lifts code quality [13].

B. Client Ranking

Table VIII reports the client scores. The Element X / Matrix Rust SDK stack wins at 9.03, leading on feature completeness, developer experience, and community, and scoring strongly on performance thanks to its native shared Rust core [15], [19]. FluffyChat is the strongest single-codebase alternative at 8.07: it tops platform coverage (9.5) by reaching all four targets from one Flutter codebase, but trails Element on feature depth and developer-tooling maturity [22]. Cinny (7.05) scores well on performance and is an excellent lightweight web client but offers only PWA mobile [23]. SchildiChat (6.93) inherits Element’s feature set and performance but lacks native iOS, capping its platform coverage [24].

C. Radar Visualisation

Figure 11 overlays the dimension profiles. The server chart shows Tuwunel enclosing every competitor except at the code-quality vertex; the client chart shows the characteristic trade-off between Element X (deep and balanced, with a slight platform-coverage notch) and FluffyChat (a platform-coverage spike with shallower feature and developer-experience radii).

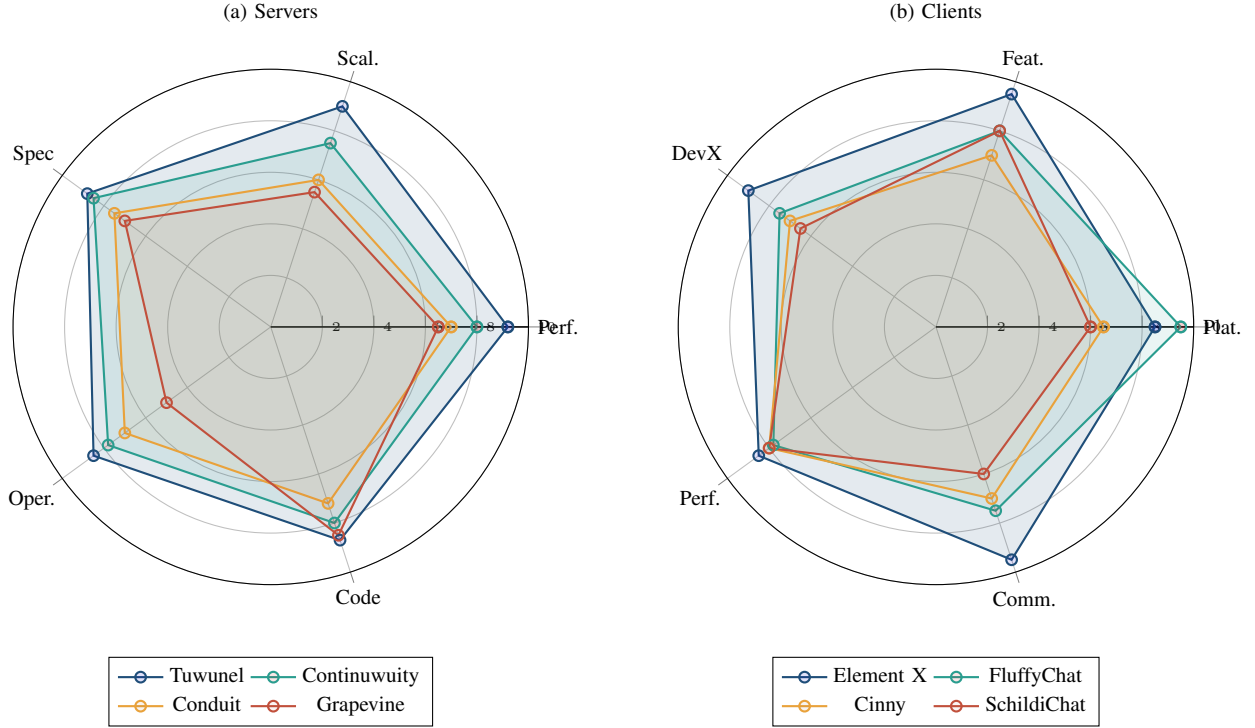


Fig. 11. Dimension radar profiles. (a) Servers: Tuwunel (blue) dominates the area, with Grapevine’s only protrusion at the code-quality axis. (b) Clients: Element X (blue) is the most balanced and feature-rich, while FluffyChat (teal) spikes on platform coverage at the cost of feature and developer-experience radius. Axes are the five scored dimensions of each class [12], [15], [22].

TABLE VIII
CLIENT RAW DIMENSION SCORES (0–10) AND WEIGHTED TOTAL S_i .
WEIGHTS FROM TABLE VI.

	Plat.	Feat.	DevX	Perf.	Comm.	S_i
Weight w_j	.20	.25	.25	.15	.15	
Element X / Rust SDK	8.5	9.5	9.0	8.5	9.5	9.03
FluffyChat	9.5	8.0	7.5	7.8	7.5	8.07
Cinny	6.5	7.0	7.0	8.0	7.0	7.05
SchildiChat	6.0	8.0	6.5	8.0	6.0	6.93

D. Statistical and Sensitivity Analysis

The winners are not artefacts of a particular weighting. For the servers, the margin between Tuwunel (8.88) and the runner-up Continuuity (8.00) is 0.88 points on a ten-point scale, and Tuwunel leads on four of five dimensions; no convex reweighting of the five dimensions can overturn a candidate that dominates coordinate-wise on all but one axis unless essentially all weight is placed on that single axis (code quality), which contradicts the product brief. The ranking is therefore robust.

For the clients, the contest is between Element X and FluffyChat, which differ only on the platform-coverage axis in FluffyChat’s favour. Element X leads on the other four dimensions, so it wins under the baseline weights and remains ahead until platform coverage is weighted far above the others. Table IX makes this concrete: re-running Eq. (1) with platform coverage raised to 0.40 (the surplus removed proportionally

TABLE IX
CLIENT SENSITIVITY TO THE PLATFORM-COVERAGE WEIGHT w_{PLAT}
(REMAINING WEIGHT SPLIT PROPORTIONALLY OVER THE OTHER FOUR
DIMENSIONS). ELEMENT X RETAINS THE LEAD ACROSS REALISTIC
WEIGHTINGS.

	$w_{\text{plat}} = 0.20$	$w_{\text{plat}} = 0.40$	$w_{\text{plat}} = 0.60$
Element X / Rust SDK	9.03	8.93	8.82
FluffyChat	8.07	8.43	8.80

from the other client weights) still leaves Element X ahead, 8.93 to 8.43; FluffyChat overtakes only when single-codebase coverage is made the dominant criterion. A team for whom one codebase across four native targets is the overriding constraint should therefore prefer FluffyChat; under balanced enterprise priorities, Element X wins.

E. Cross-Cutting Observations

Beyond the per-class rankings, three ecosystem-wide patterns emerge from placing the two surveys side by side, and each one reinforces the selections.

First, the ecosystem is *converging on a small set of audited cryptographic and protocol cores*. The winning server and client do not merely score well in isolation; they sit on shared, scrutinised foundations — the `ruma` type crates and tuned RocksDB on the server side, and the Matrix Rust SDK with `vodozamac` on the client side [17], [12], [15], [16]. Strikingly, the cryptographic convergence crosses language boundaries: the Rust and Dart client stacks both delegate end-to-end encryption to `vodozamac`, so the remediation of the

previously reported attacks propagates to clients that share almost no other code [2], [22], [16]. For a team selecting a baseline, this means a security investment in the core is amortised across the field rather than stranded in one project.

Second, *memory-safe Rust is becoming the default substrate on both sides of the protocol*. The server survey is Rust by construction, but the client survey shows the same gravitational pull: the highest-scoring client core, the next-generation Element X applications, and SchildiChat Next are all built on the Rust SDK rather than the older JavaScript or Kotlin/Swift stacks, for the same safety reasons that motivate Rust homeservers [15], [8], [24]. The implication for GaussMatrix and GaussInteract is a coherent, single-language security story spanning server and client.

Third, *project governance and continuity are first-order selection signals, not incidental metadata*. The Conduit lineage — original Conduit, the archived conduwuit, and its successors Tuwunel and Continuuity — shows that a technically strong project can be abandoned and forked within a single year, and that the surviving forks differ less in raw capability than in their backing and maintenance model [10], [12], [11]. Tuwunel’s full-time staff and national-scale production deployment are precisely the continuity guarantees an enterprise baseline needs, and they distinguish it from otherwise capable but lightly-staffed peers far more than any single feature does [14]. The same logic favours the corporately-backed Element stack on the client side.

Construct validity: the raw dimension scores synthesise documentary and source-code evidence against an anchored rubric rather than wall-clock measurements on identical hardware, which the candidates’ heterogeneous deployment models (single-process Rust binaries versus a worker-sharded Python reference server) make difficult to equalise; we mitigate this by publishing every raw score and the linear aggregation so a reader can substitute measured values. *External validity*: the Matrix ecosystem evolves rapidly, and the conduwuit-to-Tuwunel transition shows that a leading project can be archived and succeeded within a single year, so the present ranking is explicitly a Q2 2026 snapshot [12], [11]. *Internal validity*: weight selection is a value judgement, which we address by deriving the weights from the stated product briefs and by reporting the sensitivity analysis of Table IX, showing that the winners hold across realistic reweightings. None of these threats overturns the coordinate-wise dominance that drives the two selections, but each bounds how far the numeric scores should be read as precise rather than ordinal.

VIII. RECOMMENDATIONS

A. Server Baseline for GaussMatrix: Tuwunel

We recommend **Tuwunel** as the GaussMatrix baseline. It wins the benchmark (8.88) and, decisively for an enterprise product, it is the only Rust homeserver in the survey with a documented production deployment at national scale — the Government of Switzerland runs it for citizen-facing communication [14]. It is the official successor to the archived conduwuit, is maintained by full-time staff, and ships a tuned asynchronous RocksDB backend with CPU optimisations and

current room-version support [12], [14]. Its packaging breadth — static binaries plus Deb, RPM, Arch, Alpine, and Nix artefacts and OCI containers — matches enterprise operational expectations.

Integration considerations and risks. The entire Conduit family shares a single linear database schema version, so Tuwunel can adopt a conduwuit data directory by binary swap and honours the legacy `CONDUWUIT_` and `CONDUIT_` environment prefixes [12]; the same property means cross-fork migration in the *other* direction can corrupt the store and must be avoided. As a single-process server it scales primarily vertically, so a GaussMatrix deployment must plan capacity around large instances and federation sharding rather than Synapse-style worker fan-out [10], [25]. The licensing is Apache-2.0, which is permissive enough for a commercial derivative.

B. Client Baseline for GaussInteract: Element X / Matrix Rust SDK

We recommend the **Element X / Matrix Rust SDK** stack as the GaussInteract baseline, with Element Web for the browser and desktop surface. The Rust SDK provides a single, native, memory-safe core shared by the iOS and Android clients through `uniffi` bindings, delegating end-to-end encryption to the audited `vodozamac` library that remediates the cryptographic issues earlier analyses found in Matrix clients [15], [16], [2], [3]. It tops the client benchmark (9.03) on feature completeness, developer experience, and community, and carries the strongest corporate backing in the ecosystem [19].

Integration considerations and risks. Element X’s mobile clients assume a modern server with sliding-sync support; pairing GaussInteract with GaussMatrix therefore requires verifying that the chosen homeserver exposes the required sync endpoints. The codebase is AGPL/dual-licensed, so a closed derivative needs a commercial agreement, whereas FluffyChat and SchildiChat are also AGPL and Cinny is permissive MIT [23]. If single-codebase coverage of all four platforms from one team is the overriding constraint — as the sensitivity analysis in Table IX quantifies — **FluffyChat** is the recommended alternative baseline, since it reaches Android, iOS, Web, and three desktop targets from one Flutter codebase and shares the same `vodozamac` cryptographic core [22].

Figure 12 encodes the selection logic as a decision tree, so the recommendation can be re-derived for a deployment with different constraints.

C. Roadmap to GaussMatrix and GaussInteract

A pragmatic path forks each baseline rather than rebuilding it. For GaussMatrix: (i) fork Tuwunel and pin the dependency tree under `cargo deny` audit; (ii) harden the operational surface with first-party Helm charts and observability, addressing the area where lighter forks score lowest; (iii) validate CS/SS/AS and E2EE conformance against the specification test suite [1]; and (iv) plan a federation-sharding strategy to mitigate the single-process scaling ceiling [4]. For GaussInteract: (i) adopt the Matrix Rust SDK as the shared core and

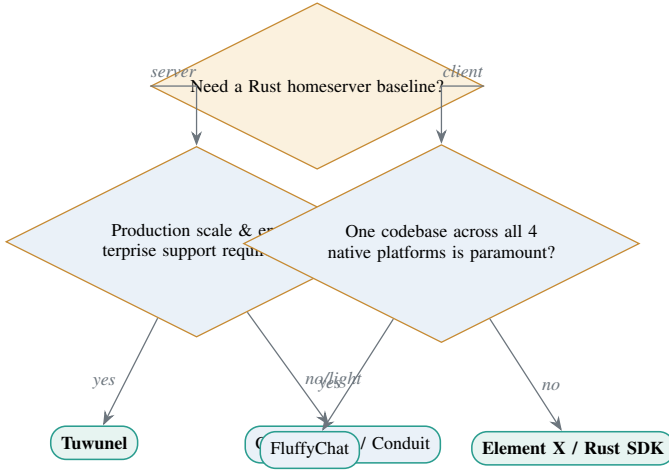


Fig. 12. Selection decision tree. The server branch resolves to Tuwunel for production enterprise scale and to the lighter Continuuity/Conduit options otherwise; the client branch resolves to FluffyChat when single-codebase native coverage dominates and to Element X / Matrix Rust SDK under balanced enterprise priorities [12], [15], [22].

the Element X applications as the mobile baseline; (ii) reuse Element Web for browser/desktop or, if a single codebase is mandated, pivot the whole baseline to FluffyChat; (iii) layer product-specific UI on the SDK while keeping *vodozamac* untouched so the audited cryptographic guarantees are preserved [16], [3]; and (iv) track upstream releases closely, since both baselines release frequently.

IX. CONCLUSION AND FUTURE WORK

This paper presented a systematic survey of open-source Matrix protocol implementations and a reproducible benchmark framework for selecting server and client baselines. Answering RQ1, we identified four actively maintained Rust homeservers meeting the inclusion criteria — Conduit, Continuuity, Tuwunel, and Grapevine — tracing their shared lineage from the original Conduit through the archived conduwuit, and analysed each one’s architecture, storage, federation design, and limitations. Answering RQ2, we surveyed four cross-platform client families — the Element family (with Element X on the Matrix Rust SDK), FluffyChat, Cinny, and SchildiChat — characterising their SDK foundations, platform coverage, encryption cores, and feature completeness. Answering RQ3, we designed a five-dimension, weighted, anchored scoring framework with an explicit aggregation model and a sensitivity analysis. Answering RQ4, the benchmark selected **Tuwunel** as the server baseline (8.88) and the **Element X / Matrix Rust SDK** stack as the client baseline (9.03), with FluffyChat as the single-codebase alternative; both recommendations were shown to be robust to realistic reweightings.

Several limitations frame the result. The dimension scores are an evidence-based synthesis of a Q2 2026 snapshot rather than wall-clock measurements on identical hardware, and the Matrix ecosystem moves quickly — forks are created, archived, and renamed within months, as the conduwuit lineage illustrates [12], [11]. Future work should (i) instrument a live testbed that drives all four servers under an identical federated workload to replace synthesised performance

scores with measured throughput and tail latency; (ii) extend the client benchmark with automated energy and cold-start profiling on real devices; and (iii) incorporate the formal-analysis literature into a continuous conformance harness so that cryptographic regressions are caught against the published attacks [2], [3], [7]. The framework itself is designed to absorb such refinements: because aggregation is linear and the inputs are documented, new measurements drop in without redesigning the model.

REFERENCES

- [1] The Matrix.org Foundation, “Matrix Specification,” 2024. [Online]. Available: <https://spec.matrix.org/>
- [2] M. R. Albrecht, S. Celi, B. Dowling, and D. Jones, “Practically-exploitable cryptographic vulnerabilities in Matrix,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2023, pp. 164–181, doi: 10.1109/SP46215.2023.10351027.
- [3] D. Jones, B. Dowling, and M. R. Albrecht, “Device-oriented group messaging: A formal cryptographic analysis of Matrix’ core,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2024.
- [4] F. Jacob, J. Grashöfer, and H. Hartenstein, “A glimpse of the Matrix: Scalability issues of a new message-oriented data synchronization middleware,” in *Proc. 20th Int. Middleware Conf. Demos and Posters*, 2019, pp. 5–6, doi: 10.1145/3366627.3368106.
- [5] F. Jacob, C. Beer, M. Henze, and H. Hartenstein, “Analysis of the Matrix event graph replicated data type,” *IEEE Access*, vol. 9, pp. 28317–28333, 2021.
- [6] H. Li, Y. Wu, T. Huang, X. Mi, X. Hu, and L. Guo, “Demystifying decentralized Matrix communication network: Ecosystem and security,” in *Proc. IEEE Int. Conf.*, 2023, IEEE Xplore no. 10476262.
- [7] J. Ginesin and C. Nita-Rotaru, “The Matrix reloaded: A mechanized formal analysis of the Matrix cryptographic suite,” arXiv:2408.12743, 2024.
- [8] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proc. ACM Program. Lang.*, vol. 2, no. POPL, art. 66, 2018.
- [9] D. J. Bernstein, “Curve25519: New Diffie-Hellman speed records,” in *Public Key Cryptography (PKC)*, 2006, pp. 207–228.
- [10] T. Kösters *et al.*, “Conduit: A Matrix homeserver written in Rust,” 2024. [Online]. Available: <https://gitlab.com/famedly/conduit>
- [11] The Continuuity contributors, “Continuuity: Community continuation of the conduwuit Matrix homeserver,” 2026. [Online]. Available: <https://forgejo.ellis.link/continuuwuation/continuuwity>
- [12] J. Volk *et al.*, “Tuwunel: High-performance Matrix homeserver (successor to conduwuit),” 2026. [Online]. Available: <https://github.com/matrix-construct/tuwunel>
- [13] The Grapevine contributors, “Grapevine: A Matrix homeserver forked from Conduit,” 2024. [Online]. Available: <https://gitlab.computer.surgery/matrix/grapevine>
- [14] The Matrix.org Foundation, “This Week in Matrix 2025-12-05,” 2025. [Online]. Available: <https://matrix.org/blog/2025/12/05/this-week-in-matrix-2025-12-05/>
- [15] The Matrix.org contributors, “matrix-rust-sdk: A Matrix client-server SDK in Rust,” 2026. [Online]. Available: <https://github.com/matrix-org/matrix-rust-sdk>
- [16] The Matrix.org contributors, “vodozamac: A Rust implementation of Olm and Megolm,” 2024. [Online]. Available: <https://github.com/matrix-org/vodozamac>
- [17] The Ruma contributors, “Ruma: Matrix protocol types and APIs for Rust,” 2024. [Online]. Available: <https://github.com/ruma/ruma>
- [18] The Tokio contributors, “Tokio: An asynchronous runtime for Rust,” 2024. [Online]. Available: <https://tokio.rs/>
- [19] Element (New Vector Ltd.), “element-web: A glossy Matrix client,” 2026. [Online]. Available: <https://github.com/element-hq/element-web>
- [20] Element (New Vector Ltd.), “element-x-ios: Next-generation Matrix iOS client,” 2026. [Online]. Available: <https://github.com/element-hq/element-x-ios>
- [21] Element (New Vector Ltd.), “element-x-android: Next-generation Matrix Android client,” 2026. [Online]. Available: <https://github.com/element-hq/element-x-android>
- [22] K. Krille *et al.*, “FluffyChat: A cross-platform Matrix client built with Flutter,” 2026. [Online]. Available: <https://github.com/krille-chan/fluffychat>

- [23] A. Yadav *et al.*, “Cinny: A Matrix client focusing on simplicity and elegance,” 2026. [Online]. Available: <https://github.com/cinnyapp/cinny>
- [24] The SchildiChat contributors, “SchildiChat: A Matrix client forked from Element,” 2026. [Online]. Available: <https://github.com/SchildiChat>
- [25] Element (New Vector Ltd.), “Synapse: Matrix reference homeserver,” 2026. [Online]. Available: <https://github.com/element-hq/synapse>
- [26] The Matrix.org contributors, “Dendrite: A second-generation Matrix homeserver in Go,” 2024. [Online]. Available: <https://github.com/matrix-org/dendrite>
- [27] The Matrix.org Foundation, “Matrix ecosystem: Servers,” 2026. [Online]. Available: <https://matrix.org/ecosystem/servers/>
- [28] The Matrix.org Foundation, “Matrix ecosystem: Clients,” 2026. [Online]. Available: <https://matrix.org/ecosystem/clients/>
- [29] M. Hodgson and A. Le Pape, “Matrix: An open standard for decentralised communication,” The Matrix.org Foundation, 2014. [Online]. Available: <https://matrix.org/>